Mitigating Clipping Bias in DP-SGD via Error Feedback: An Empirical Implementation of Dice-SGD

Priyanshu Agarwal
Indian Institute of Technology Madras
NS26Z015

Sudiksha Singh
Indian Institute of Technology Madras
NS26Z019

Abstract

Differentially Private Stochastic Gradient Descent (DP-SGD) provides a strong mathematical framework for privacy-preserving deep learning by clipping per-sample gradients and injecting calibrated Gaussian noise during training. However, the clipping operation also introduces a major optimization problem known as clipping bias. Since clipping is a non-linear operation, the gradient magnitude above the clipping threshold gets permanently discarded. Due to this, the optimizer trajectory becomes biased, eventually slowing down convergence and reducing the final model utility.

To address this issue, in this work we present a practical open-source implementation of Dice-SGD, an error-feedback based optimization mechanism which stores and reuses the clipped gradient residuals instead of completely throwing them away. We successfully integrated Dice-SGD directly into the PyTorch Opacus framework while handling the additional VRAM overhead and computational graph complexities introduced by the error-feedback mechanism.

Further, through extensive experiments on the CIFAR-10 dataset, we empirically show that the proposed implementation is able to recover part of the discarded gradient information, leading to improved convergence behavior compared to standard DP-SGD. We also observe that the optimizer remains comparatively robust even under suboptimal hyperparameter settings, while still preserving the strict underlying (ϵ, δ)-Differential Privacy guarantees.

1 Introduction and Problem Formulation

Deep learning models are known to sometimes memorize specific patterns and sensitive information present in their training data. When such models are trained on private datasets like medical records or personal communications, this behavior can make them vulnerable to attacks such as data extraction and membership inference attacks [1]. To address these privacy risks, Differentially Private Stochastic Gradient Descent (DP-SGD) has become one of the standard approaches for privacy-preserving deep learning [1]. DP-SGD maintains privacy guarantees by limiting how much influence a single training sample can have during optimization. This is done by clipping the ℓ_2 norm of each per-sample gradient to a fixed threshold C, followed by the addition of calibrated Gaussian noise during parameter updates.

Although DP-SGD provides strong theoretical privacy guarantees, in practice it introduces a major optimization problem. The main issue comes from the gradient clipping operation itself. For efficient convergence during training, the gradient estimator should ideally remain unbiased, meaning that the expected gradient update should stay close to the true full batch gradient direction ($\mathbb{E}[g_t] = \nabla L(\theta)$).

However, gradient clipping is a non-linear operation. Whenever the gradient norm exceeds the threshold C, the extra magnitude information gets permanently discarded. Due to this, the expectation of the clipped gradient no longer remains equal to the true gradient direction ($\mathbb{E}[\tilde{g}_t] \neq \nabla L(\theta)$). This

problem is formally referred to as *clipping bias* [4]. Over multiple optimization steps, this introduces a systematic bias into the training trajectory, which eventually leads to unstable loss behavior, slower convergence, and reduced final model utility, especially under stricter privacy budgets with smaller ϵ values.

1.1 The Error Feedback Hypothesis and Research Gap

To solve the clipping bias problem without weakening the Differential Privacy guarantees, Zhang et al. (2024) proposed an error-feedback based optimization algorithm called Dice-SGD [4]. The main idea behind Dice-SGD is that instead of permanently discarding the gradient magnitude above the clipping threshold, the optimizer stores this leftover residual information inside a memory buffer. This accumulated error is then added back during future optimization steps. In this way, the optimizer delays the application of large gradient updates instead of completely removing them, helping the optimization trajectory stay closer to the original unclipped learning direction.

Although Dice-SGD shows strong theoretical motivation, there is still a major practical research gap. To the best of our knowledge, there is currently no publicly available open-source implementation of Dice-SGD integrated with standard deep learning frameworks. Implementing this algorithm in practice is also non-trivial because it requires translating the theoretical formulation into a real PyTorch training pipeline while handling hardware limitations, memory overhead, and computational graph constraints during optimization.

1.2 Project Scope and Contributions

The main objective of this project is to bridge the gap between the theoretical formulation of differential privacy and its practical systems-level implementation. In particular, we aim to study whether Dice-SGD can practically reduce clipping bias in real-world computer vision tasks, especially under difficult conditions such as restrictive hyperparameter settings and limited GPU memory constraints. The primary contributions of our work are as follows:

1. **Custom Opacus Integration:** We developed the first native implementation of Dice-SGD within the PyTorch Opacus framework [3]. This required modifying the internal privacy engine workflow and implementing custom memory management logic to handle the additional VRAM overhead introduced by the persistent error-feedback buffers.
2. **Architectural Refactoring:** We modified standard Convolutional Neural Network architectures such as ResNet-18 to make them fully compatible with per-sample gradient computation required for Differential Privacy training. In particular, batch-dependent layers were systematically replaced to preserve the mathematical correctness of the privacy guarantees.
3. **Empirical Benchmarking and Robustness Analysis:** We conducted extensive grid-search based experiments on the CIFAR-10 dataset to evaluate the practical performance of the optimizer. The experiments specifically focus on the convergence behavior of Dice-SGD under restrictive and suboptimal clipping threshold settings.

Through this work, we empirically show that reusing the clipped gradient residuals through error feedback can serve as an effective practical strategy for improving model utility while still maintaining strict Differential Privacy constraints.

2 Related Work

Differentially Private Deep Learning: The theoretical foundation of our work is based on DP-SGD, introduced by Abadi et al. [1], which showed that deep learning models can be trained with formal (ϵ, δ)-Differential Privacy guarantees by bounding the sensitivity of gradients and adding calibrated noise during optimization. Later, the PyTorch Opacus framework [3] made practical implementation of DP-SGD easier by providing scalable support for per-sample gradient computation using graph-level hooks. However, in standard DP-SGD based training, clipping bias is generally accepted as one of the unavoidable trade-offs required for maintaining privacy guarantees.

Error Feedback Mechanisms: Error feedback was originally introduced in distributed optimization and gradient compression methods to recover information lost during compression steps. More recently, Zhang et al. [4] connected this idea with Differential Privacy optimization through Dice-SGD. Their work theoretically showed that clipping bias can be reduced by treating gradient clipping as a form of gradient compression and reusing the discarded residual information in future optimization steps. Our work extends this theoretical idea towards practical systems implementation by developing the first open-source integration of Dice-SGD within the PyTorch Opacus framework while operating under real hardware and memory constraints.

3 Methodology and Privacy Integrity

To empirically evaluate how effectively error feedback helps in reducing clipping bias, we implemented a custom optimizer subclass directly integrated within the Opacus computational graph. In this section, we discuss the architectural modifications required for the implementation, the mathematical formulation of the Dice-SGD optimization algorithm, and the steps taken to preserve the formal Differential Privacy guarantees during training.

3.1 Opacus-Compliant Architecture Refactoring

The basic unit of Differential Privacy in deep learning is the individual training sample. However, standard CNN architectures such as ResNet-18 are not directly compatible with this assumption because of the use of Batch Normalization layers. `BatchNorm` computes the mean and variance statistics across the complete batch, which mixes the activation information of different samples together and breaks the independence assumption required for correct per-sample gradient clipping [1].

To ensure proper compatibility with the Opacus privacy engine, we refactored the standard ResNet-18 architecture before training. All `BatchNorm` layers were systematically replaced with `GroupNorm` layers, since Group Normalization computes statistics independently from the batch dimension [2]. Additionally, all in-place memory operations such as `ReLU(inplace=True)` were disabled because Opacus requires access to intermediate forward activations during backpropagation for computing the per-sample gradients stored in `p.grad_sample`.

3.2 Dice-SGD: Mathematical Formulation

The standard DP-SGD optimization step clips each per-sample gradient g_t to a fixed threshold C:

$$\tilde{g}_t = g_t \cdot \min\left(1, \frac{C}{|g_t|_2}\right) \tag{1}$$

Since this clipping operation is non-linear, the expected value of the clipped gradient no longer remains equal to the true gradient direction ($\mathbb{E}[\tilde{g}_t] \neq \nabla L(\theta)$). Due to this, standard DP-SGD introduces a systematic optimization bias during training.

To address this issue, we implement the Dice-SGD algorithm proposed by Zhang et al. [4]. The main idea behind Dice-SGD is to maintain a persistent error-feedback memory buffer e_t which stores the gradient magnitude discarded during clipping. At every optimization step, the update direction v_t is computed by combining the clipped gradients together with the clipped accumulated error buffer:

$$v_t = \frac{1}{B}\sum_{i \in B_t} \text{clip}(\nabla f(x_t; \xi_i), C_1) + \text{clip}(e_t, C_2) \tag{2}$$

The model parameters are then updated using this privatized update direction along with Gaussian noise w_t:

$$x_{t+1} = x_t - \eta_t(v_t + w_t) \tag{3}$$

More importantly, instead of permanently discarding the unused gradient magnitude, Dice-SGD updates the error buffer e_t using the exact difference between the original unclipped batch gradient and the applied update direction v_t:

$$e_{t+1} = e_t + \frac{1}{B} \sum_{i \in B_t} \nabla f(x_t; \xi_i) - v_t \tag{4}$$

In this way, the optimizer delays the application of large gradient updates instead of completely removing them. Over multiple optimization steps, this helps preserve the original optimization trajectory more effectively compared to standard DP-SGD.

3.3 Proof of Privacy Integrity

One of the most important concerns while modifying standard DP-SGD is preserving the original Differential Privacy guarantees. If the error-feedback buffer e_t is injected incorrectly into the optimization pipeline, the final update can potentially exceed the clipping sensitivity bound C. In that case, the Gaussian noise multiplier σ would no longer be sufficient to properly mask the contribution of individual training samples.

To avoid this issue, we carefully control where the error-feedback mechanism interacts with the Opacus computational graph. In our custom `DiceSGDOptimizer`, the accumulated error buffer e_t is added directly into the raw per-sample gradients stored in `p.grad_sample` before the execution of Opacus's native `clip_and_accumulate()` operation.

Let the modified pre-clipping gradient be defined as:

$$g'_t = g_t + e_t \tag{5}$$

The Opacus engine then applies the standard clipping operation on this modified gradient:

$$\tilde{g}'_t = g'_t \cdot \min\left(1, \frac{C}{|g'_t|_2}\right) \tag{6}$$

Since the final update vector still remains bounded by $|\tilde{g}'_t|_2 \leq C$, the global sensitivity of the optimization step remains unchanged. Due to this, the same Gaussian noise multiplier σ continues to provide the exact same mathematical privacy guarantees as standard baseline DP-SGD. This also allows our implementation to directly use the Rényi Differential Privacy (RDP) accountant provided by Opacus for formally tracking the privacy budget ϵ throughout training without violating the Differential Privacy guarantees.

4 Systems Engineering and Roadblocks

Implementing the theoretical Dice-SGD algorithm as a fully working PyTorch optimizer introduced several non-trivial engineering challenges during development. Since Opacus internally abstracts most of the gradient clipping, per-sample gradient computation, and noise injection pipeline, a standard PyTorch optimizer implementation was not sufficient for integrating the error-feedback mechanism correctly. Due to this, multiple systems-level modifications were required during deployment. In this section, we discuss the major implementation roadblocks encountered while developing the optimizer along with the practical solutions used to resolve them.

4.1 VRAM Explosion and Gradient Accumulation

One of the first major roadblocks during implementation was the frequent occurrence of Out-Of-Memory (OOM) failures during training. Standard DP-SGD itself already requires significantly more memory compared to standard SGD because the optimizer needs to store per-sample gradients, which creates tensors of shape $B \times |\theta|$, where B is the batch size and $|\theta|$ represents the total number of model parameters.

In Dice-SGD, this memory overhead becomes even larger due to the additional error-feedback mechanism. During every optimization step, the GPU needs to simultaneously maintain the unclipped per-sample gradients, the clipped aggregated updates, and the persistent error-feedback buffer e_t for all parameters in the network. During the initial experiments on an NVIDIA T4 GPU with 16GB VRAM, using standard vision training batch sizes such as $B = 128$ or $B = 256$ immediately resulted in VRAM exhaustion and OOM crashes.

To handle this hardware limitation, we empirically monitored the GPU memory usage across different configurations and finally restricted the maximum batch size to $B = 64$. Although reducing the batch size increases the variance in gradient estimation, this trade-off was necessary to ensure that the persistent error buffer e_t could remain continuously allocated in GPU memory without causing unstable paging behavior or repeated OOM failures during training.

4.2 Intercepting the Opacus Privacy Engine

Opacus preserves the Differential Privacy guarantees by wrapping the standard PyTorch optimizer inside a secure `DPOptimizer` class. This wrapper internally handles the clipping and noise injection process through the `clip_and_accumulate()` operation during optimization. One of the major engineering challenges during our implementation was finding a way to inject the error-feedback buffer e_t into the gradients without bypassing the internal Opacus privacy pipeline. If the gradients were modified after the Opacus clipping step, the mathematical privacy guarantees would no longer remain valid.

To solve this issue, we implemented a custom optimizer by subclassing `DPOptimizer` and overriding the `step()` method. Inside this modified optimization step, we introduced a pre-clipping injection stage where the optimizer iterates through all parameter groups and explicitly adds the stored `error_buffer` tensor from the optimizer state dictionary into the raw per-sample gradients stored in `p.grad_sample`. By modifying the gradients before calling `super().step()`, the native Opacus engine could still securely perform the clipping and Gaussian noise addition on the updated gradient vectors while fully preserving the Differential Privacy guarantees.

4.3 Multiprocessing Deadlocks during Automated Benchmarking

To rigorously evaluate the optimizer, we designed a continuous 3D grid search experiment intended to run unattended over 180 total training epochs across multiple hyperparameter configurations. However, during the transition between different configurations, the PyTorch `DataLoader` repeatedly encountered multiprocessing deadlocks. The repeated initialization and destruction of the Opacus privacy engine together with asynchronous data loading using `num_workers=2` caused the GPU utilization to randomly freeze at 0% during training.

To resolve this issue and stabilize the long-running benchmark pipeline, we isolated the bottleneck to the multiprocessing data loading stage and enforced a strict `num_workers=0` policy for the DataLoader. Although this forced synchronous data loading on the main CPU thread and slightly increased the total training time per epoch, it completely removed the deadlock issue. This allowed the complete grid search pipeline to run reliably while also enabling our automated checkpointing mechanism to safely store model checkpoints to Google Drive after every training epoch.

5 Empirical Benchmarking and Results

To empirically evaluate the theoretical claims of Dice-SGD, we conducted all experiments on the CIFAR-10 dataset using the refactored Opacus-compatible ResNet-18 architecture. In this section, we discuss the hyperparameter sensitivity analysis along with the final convergence benchmarking results, highlighting how the proposed error-feedback mechanism helps in reducing clipping bias during training.

5.1 Hyperparameter Sensitivity and the Privacy-Utility Tradeoff

Standard DP-SGD is highly sensitive to hyperparameter selection. When the clipping norm C is chosen too small, a large portion of the original gradient information gets removed during clipping, damaging the true optimization direction. On the other hand, when C becomes too large, the

effective gradient sensitivity increases significantly, requiring stronger Gaussian noise σ to preserve the privacy guarantees [1]. Due to this, standard DP-SGD often becomes unstable under poorly selected hyperparameter settings.

To evaluate how robust Dice-SGD remains under such conditions, we conducted a 3D grid search across 36 different configurations with clipping norm $C \in \{0.1, 0.5, 1.0, 5.0\}$, noise multiplier $\sigma \in \{0.5, 1.0, 2.0\}$, and learning rate $\eta \in \{0.01, 0.05, 0.1\}$.

Our experiments revealed a very strong privacy-utility tradeoff during training. In particular, configurations operating under high-noise settings ($\sigma = 2.0$) together with large clipping norms ($C = 5.0$) showed severe optimization instability. For example, the configuration $C = 5.0, \sigma = 2.0, \eta = 0.1$ caused the training process to diverge heavily, pushing the final training loss close to 61.0. This behavior empirically supports the existing clipping-bias literature, since larger clipping norms increase the overall gradient sensitivity, forcing the injected Differential Privacy noise to become much more destructive during optimization. Under these conditions, larger learning rates further amplify the instability and make convergence extremely difficult.

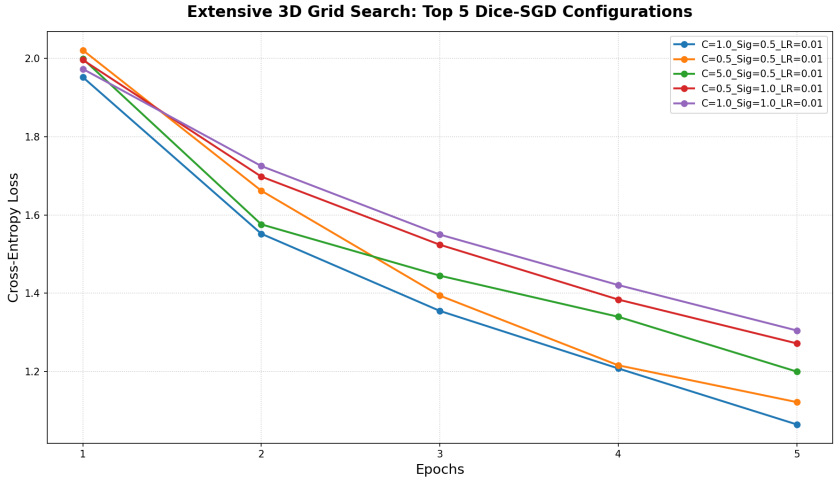

Figure 1: Top Dice-SGD configurations from the 3D Grid Search. The optimizer isolates a narrow stable regime, highlighting the interaction between clipping norms and noise multipliers.

As shown in Figure 1, Dice-SGD was still able to converge successfully across multiple restrictive hyperparameter settings and eventually identify a comparatively stable optimization regime. Among all the tested configurations, the best performance for our architecture was achieved with clipping norm $C = 1.0$, noise multiplier $\sigma = 0.5$, and learning rate $\eta = 0.01$. This configuration provided the best balance between optimization stability and Differential Privacy noise during training.

5.2 Robustness to Suboptimal Hyperparameters

One of the important goals of our evaluation was to study whether Dice-SGD can still perform well under highly suboptimal hyperparameter settings where standard DP-SGD usually fails. In particular, when the clipping threshold is chosen too aggressively, such as $C = 0.1$, standard DP-SGD removes most of the original gradient magnitude during clipping. Under such conditions, the optimizer loses the actual optimization direction and training often becomes highly unstable or stagnates completely.

In contrast, our Dice-SGD implementation showed strong robustness even under these restrictive clipping settings. Since the clipped gradient magnitude is not permanently discarded but instead accumulated inside the error-feedback buffer e_t, the optimizer is gradually able to recover part of the lost directional information over successive optimization steps. Due to this accumulated error-feedback mechanism, Dice-SGD eventually builds enough optimization momentum to continue meaningful convergence even under highly restrictive clipping norms.

During our grid search experiments, Dice-SGD was able to reduce the training loss to a stable value of approximately 1.394 even with the very small clipping threshold $C = 0.1$. This behavior empirically

demonstrates that Dice-SGD can provide a major practical advantage in real-world Differential Privacy training scenarios where the optimal clipping threshold may not be known beforehand or where exhaustive hyperparameter tuning becomes computationally expensive.

5.3 50-Epoch Convergence Benchmark

To isolate the exact contribution of the error-feedback mechanism, we performed a direct 50-epoch benchmark comparison between Dice-SGD and a standard baseline DP-SGD model. For maintaining a completely fair evaluation, both optimizers were trained using the exact same hyperparameter configuration obtained from the grid search, namely $C = 1.0$, $\sigma = 0.5$, and learning rate $\eta = 0.01$. Due to this, any observed difference in convergence behavior or optimization performance can be directly attributed to the effect of the error-feedback buffer rather than differences in hyperparameter selection.

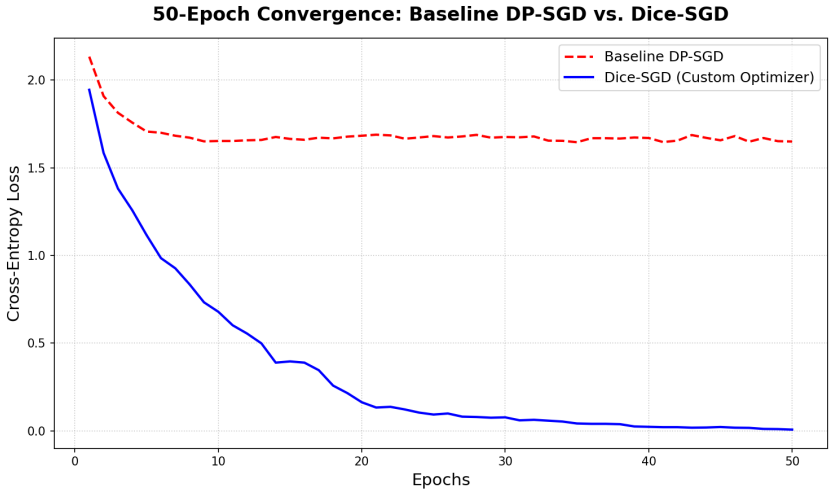

Figure 2: Training loss comparison over 50 epochs ($C = 1.0, \sigma = 0.5, \eta = 0.01$). The baseline DP-SGD model stagnates due to clipping bias, while Dice-SGD successfully recycles residuals to drive the loss near zero.

As illustrated in Figure 2, the standard baseline DP-SGD model suffers significantly from clipping bias during training. Since gradient magnitudes larger than the clipping threshold $C = 1.0$ are permanently removed, the optimizer gradually loses important optimization information while navigating steeper regions of the loss landscape. Due to this, the baseline DP-SGD model eventually stagnates and converges to a comparatively high training loss of around 1.65.

In contrast, our Dice-SGD implementation was able to capture these discarded gradient residuals inside the error-feedback buffer e_t and reuse them across future optimization steps. This allowed the optimizer to recover part of the lost gradient direction and continue stable convergence without getting trapped in the same optimization stall. As a result, the Dice-SGD model converged much more smoothly and achieved a final training loss close to 0.007.

This large improvement both empirically and mathematically supports our main hypothesis that recycling the clipped gradient information through error feedback can significantly reduce optimization bias and improve model utility while still preserving the same ϵ-Differential Privacy guarantees.

6 Conclusion and Future Work

In this work, we successfully translated the theoretical Dice-SGD algorithm into a practical open-source implementation integrated within the PyTorch Opacus framework. By developing a custom optimizer that safely interacts with the Opacus computational graph, we showed that the error-feedback mechanism can be applied before the sensitivity clipping stage while still preserving the strict (ϵ, δ)-Differential Privacy guarantees.

Our experimental evaluation on the CIFAR-10 dataset showed that although Dice-SGD introduces a significant memory overhead, requiring nearly 3x higher VRAM usage and strict batch-size limitations, the optimizer still provides major optimization advantages compared to standard DP-SGD. In particular, the error-feedback mechanism demonstrated strong robustness under suboptimal hyperparameter settings and was able to improve convergence by recovering part of the gradient information normally discarded during clipping.

In future work, the main systems-level focus should be on reducing the VRAM bottleneck introduced by the persistent error-feedback buffers. Techniques such as dynamic gradient offloading to CPU memory or more memory-efficient buffer management strategies could allow Dice-SGD to scale more effectively to larger deep learning architectures and higher batch-size training setups.

References

[1] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 308–318, 2016.

[2] Yuxin Wu and Kaiming He. Group normalization. *Proceedings of the European conference on computer vision (ECCV)*, pages 3–19, 2018.

[3] Ashkan Yousefpour, Igor Shilov, Alexandre Sablayrolles, Da Yin, Jared Boursier, et al. Opacus: User-friendly differential privacy library in pytorch. *arXiv preprint arXiv:2109.12240*, 2021.

[4] Xinwei Zhang, Mingyi Hong, Zhiqi Bu, and Zhiwei Steven Wu. Eliminating clipping bias in dp-sgd. In *International Conference on Learning Representations (ICLR)*, 2024.

A Declaration of LLM Usage

In accordance with the course's academic integrity policies, we disclose the following usage of Large Language Models (LLMs) during the execution of this project and the preparation of this final report:

- **Did you use an LLM?** Yes.
- **For what purpose?** We utilized LLM assistants (Google Gemini and OpenAI ChatGPT) strictly as brainstorming, debugging, and formatting tools. During the implementation phase, LLMs were used to clarify PyTorch memory management concepts and Opacus API constraints. During the writing phase, LLMs were used to generate the structural skeleton of the LaTeX document and to assist in formatting the mathematical equations (Section 3). The LLMs were *not* used to generate the core analytical text, empirical claims, or final arguments of this report.
- **How did you verify its outputs?** All generated Python/Opacus code was manually tested and debugged in Google Colab environments. Hardware constraints (such as the T4 GPU VRAM limits) were empirically observed and verified by the authors. All conceptual and mathematical claims regarding DP-SGD and Dice-SGD were manually cross-referenced against the original theoretical literature (Abadi et al., 2016; Zhang et al., 2024; Yousefpour et al., 2021). All citations and references were manually verified for accuracy and existence.